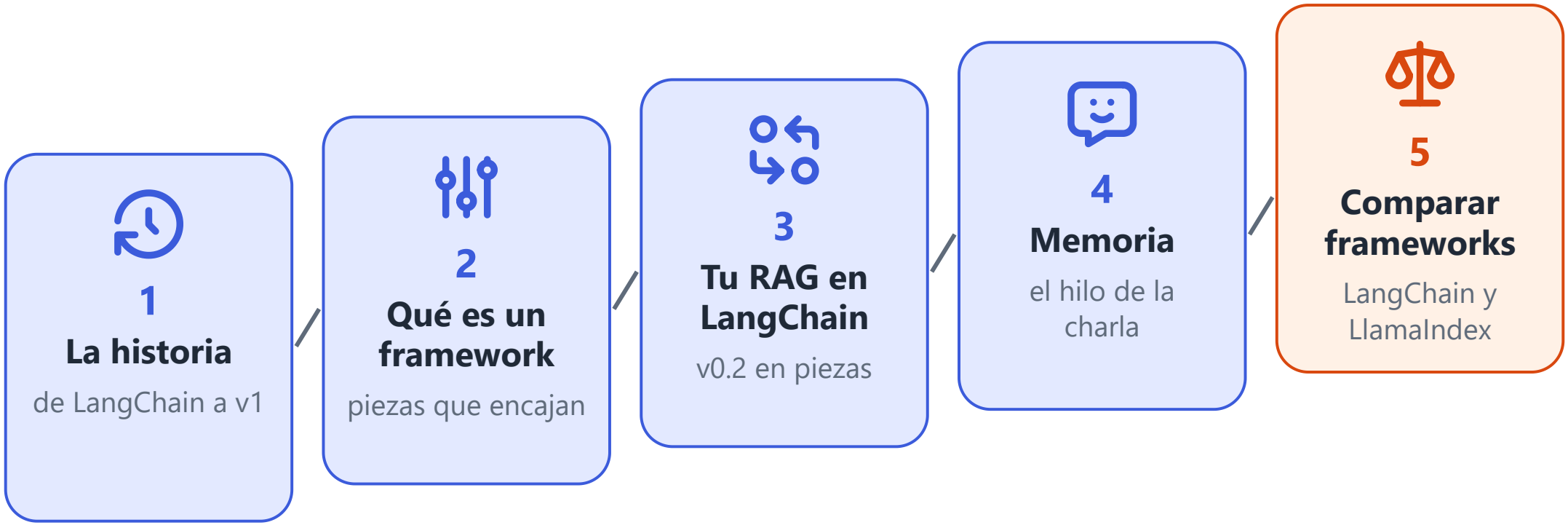
LangChain y memoria

M5: v0.3, LangChain y memoria



Hoy: el recorrido

De código a mano a piezas armadas



Venís de v0.2, armado a mano. Hoy lo pasás a piezas que encajan y le das memoria.

Hoy cada ejemplo corre en vivo: cuaderno de la clase 4.

Primero solo, después lo discutimos

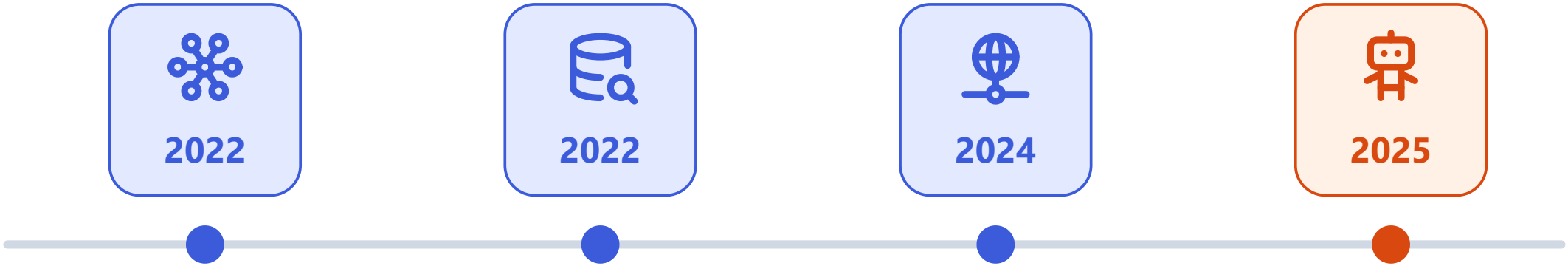
- Control de lectura individual, 5 min
- Discusión de las respuestas, 5 min

Sin asistentes de IA. 5 preguntas sobre frameworks, memoria y las lecturas. En la discusión, las mismas.

¿Para qué te sirve? Un asistente que sigue la charla

Dónde	Pasa	Lo resuelve
BI	«¿Y por región?» tras el churn	Memoria
Soporte	Cada cliente, su charla	Un hilo por cliente
Equipo	Cambiar Gemini por OpenAI	El framework
Producto	Sumarle herramientas	Piezas que encajan

Los frameworks crecieron junto con ChatGPT



Cambian rápido: el proyecto fija las versiones exactas.

Una librería da ladrillos, un framework, un kit

Librería
funciones sueltas que llamás cuando querés

→

Framework
piezas que encajan y una forma de armarlas

Con el kit vas más rápido, pero no ves todo lo que hace por dentro.

Un framework te da piezas y te esconde pasos

Te da

piezas que encajan · memoria y estado · cambiar de proveedor

Te esconde

prompts internos · llamadas de más · el costo de cada paso

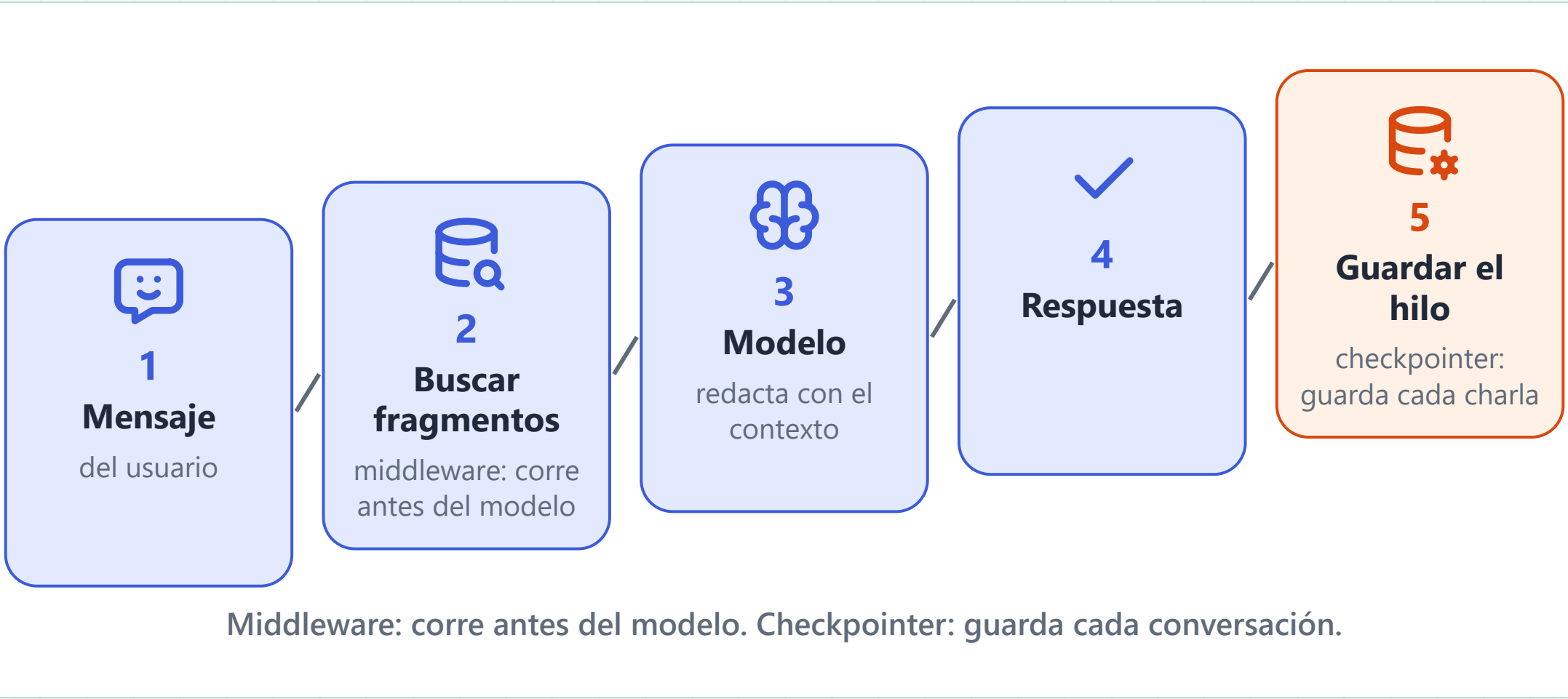
Tu v0.3 se dibuja «inicio → model → fin»: la búsqueda no se ve.

LangChain no es el único kit

- **LangChain**
- **LlamaIndex (demo de hoy)**
- **Haystack**
- **DSPy**
- **Semantic Kernel (Microsoft)**
- **Hugging Face**

Las piezas se parecen: modelo, buscador, prompt, memoria. Cambian los nombres y lo que esconden.

Tu RAG, armado con piezas de LangChain



Busca antes de redactar. Los *document loaders* leen tus archivos.

Una función que corre antes del modelo

```
@dynamic_prompt                                          — Antes de cada llamada
def con_contexto(pedido) -> str:                         — Arma el pedido
    hits = buscar(pedido.messages[-1].text)              — Tu búsqueda, con el último mensaje
    return f"{SISTEMA}…{contexto}"                       — Suma el contexto
create_agent(…, middleware=[con_contexto],               — El RAG, como middleware
             checkpointer=InMemorySaver())               — Guarda cada hilo
responder(pregunta, hilo)  # thread_id = hilo           — Un hilo por conversación
```

Busca, arma el contexto y guarda cada hilo. En vivo: sección 1.

LCEL: el mismo pedido, como una cadena con |

```
humano = "Contexto:\n{contexto}\n\nPregunta: {pregunta}"
prompt = ChatPromptTemplate.from_messages([("system", SISTEMA), ("human", humano)])
cadena = prompt | cargar_modelo() | StrOutputParser()  # cada | pasa su salida
respuesta = cadena.invoke({"contexto": contexto, "pregunta": pregunta})
```

- Como los pasos de Power Query: cada uno recibe la salida del anterior.
- Pasos fijos encadenados: eso es LCEL.

LangChain no empeoró la suite

v0.2, Chroma
hit 10/11

=

v0.3, LangChain
hit 10/11, mismos fragmentos

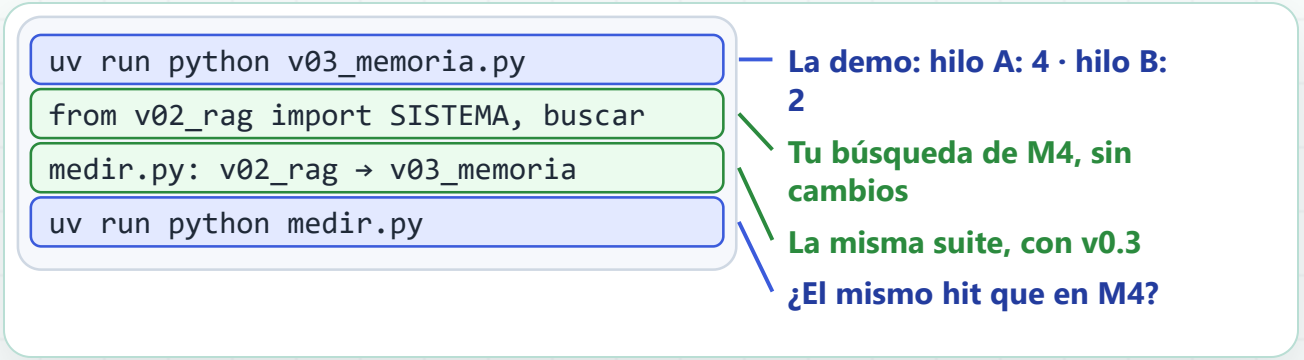
Dos corridas (15/10 y 17/10): **sin regresión**. En vivo: sección 3.

Taller: tu RAG en LangChain, medido (50 min)

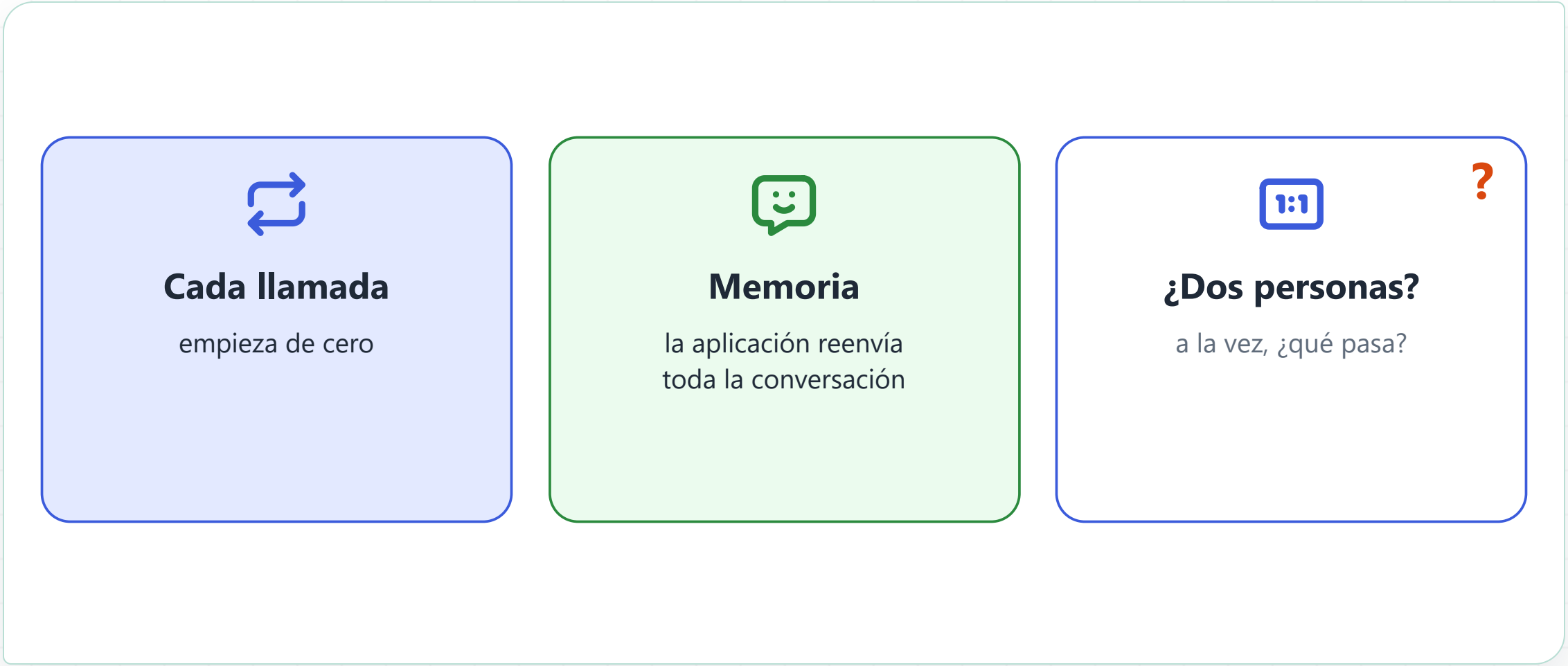
1. **Escribí** `v03_memoria.py`
2. **Medí** con `medir.py`: `v0.2` y `v0.3`
3. **Cada diferencia:** ¿otra respuesta?

Práctica M5, arranque y «En clase», pasos 1 a 3.

Primer control · 10–15 min: la demo termina en «hilo A: 4, hilo B: 2».



La API no recuerda nada entre pedidos



¿Qué pasa si dos personas la usan a la vez?

Cada conversación guarda solo lo suyo

Hilo A
2 preguntas
4 mensajes guardados

≠

Hilo B
1 pregunta
2 mensajes guardados

- **Hilo** = una conversación con su historia: como filtrar los mensajes por hilo.
- ¿Eso es memoria?

Dos corridas: el seguimiento acertó una vez

```
corrida 1  Hilo A (seguimiento): NO ENCONTRADO
           Hilo B:               Si el proceso se reinicia, todos los puntos de
                                 control … se pierden [langgraph__persistence].
corrida 2  Hilo A (seguimiento): Si se utiliza MemorySaver o InMemorySaver,
                                 todos los puntos de control se pierden…
           Hilo B:               Cuando el proceso se reinicia, … se pierden…
las dos    Mensajes guardados → hilo A: 4 · hilo B: 2
```

- Probalo: cuaderno en vivo, sección 5
- La prueba de memoria es la cuenta de mensajes.

Taller: memoria con tus preguntas (35 min)

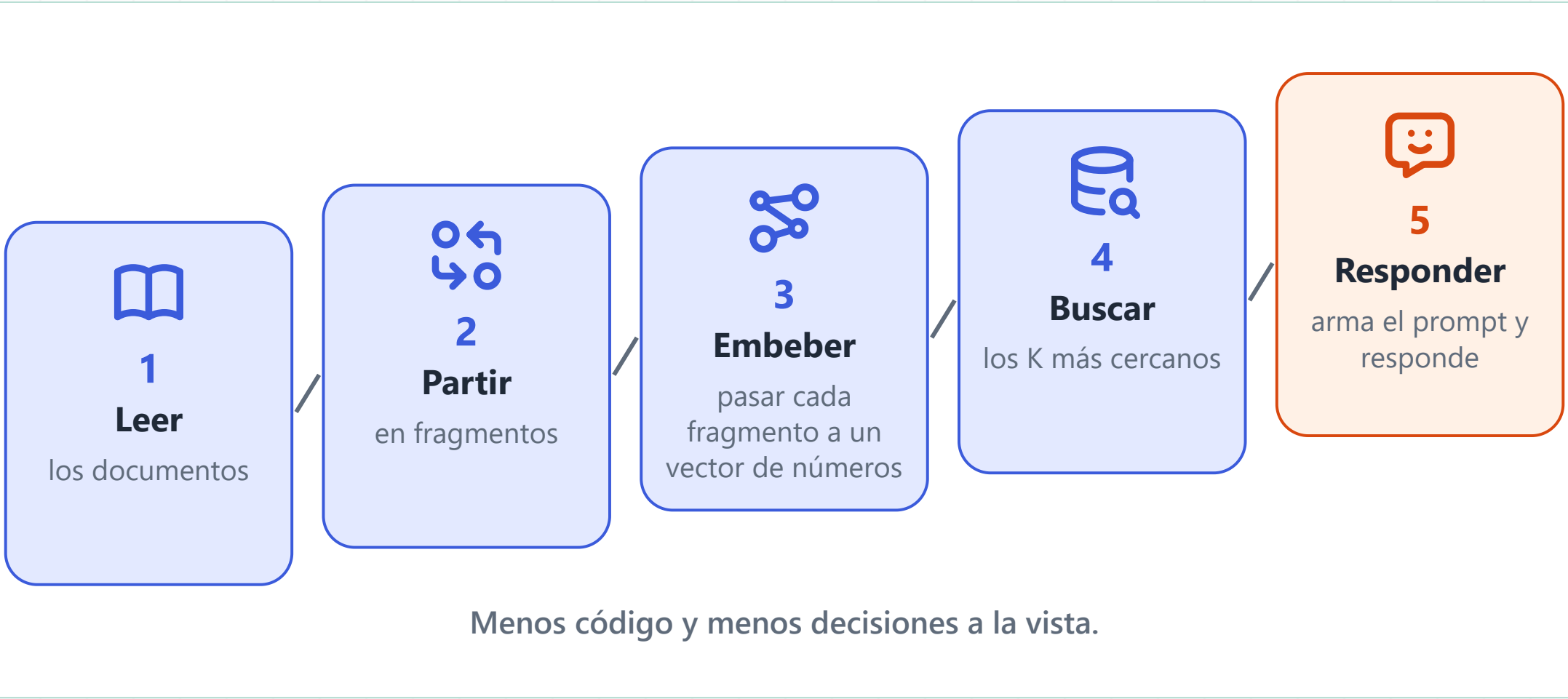
1. **Tu conversación**, con un seguimiento.
2. **Aislamiento:** B no guarda lo de A.
3. **Para casa:** ¿últimos mensajes o resumen?

Práctica M5, «En clase», pasos 4 y 5.

Trampa: la búsqueda ve solo el último mensaje.

```
responder("¿Qué guarda…?", "A")      — Tu pregunta, en el hilo A
responder("¿Y qué pasa…?", "A")      — Un seguimiento que la necesite
responder("¿Y qué pasa…?", "B")      — El mismo seguimiento, en B
mensajes_guardados("B")              — La prueba: B no guarda lo de A
```

LlamaIndex hace solo lo que en v0.2 escribiste



La demo muestra qué decidió por vos.

El mismo RAG en LlamaIndex son tres líneas

```
docs = SimpleDirectoryReader("corpus").load_data()          # leer
indice = VectorStoreIndex.from_documents(docs)              # partir y vectorizar
r = indice.as_query_engine(similarity_top_k=4).query(p)     # buscar y responder
```

- Tu `v02_rag.py` hace lo mismo a la vista.
- Ojo: de fábrica usan OpenAI y leen todo `corpus/`, licencia incluida. La demo antes les dice qué modelo y qué archivos usar.

LlamaIndex respondió largo y sin citar

```
Pregunta: ¿Qué hace interrupt en LangGraph?

── Tu v0.2 · responder (2.3 s)
`interrupt` pausa la ejecución de un grafo en un nodo … y espera una entrada
externa antes de continuar [langgraph__interrupts]. Al llamarlo, … guarda el estado

── LlamaIndex (35.3 s)
Basándose en la información provista, la función `interrupt()` … permite pausar
… 1. **Pausa la ejecución** … 2. **Guarda el estado** … 3. … 4. …   (sin citas)
fuentes: […] · fragmentos: 50
```

- Probalo: cuaderno en vivo de la clase 5, sección 1
- Los 35 s incluyen rehacer el índice. v0.2 lo reusa.

Lo que el framework decide por vos

	v0.2	LlamaIndex, de fábrica
Fragmento	1000 caracteres	1024 tokens
k	4	2
Prompt	Tu `SISTEMA` : citar	El suyo, oculto
Índice	En `.chroma`	Se rehace cada vez

Todo se cambia, pero primero hay que saber que existe.

Comparemos con una pregunta tuya

- Dictá una pregunta de tu tema: la corro en los dos.
- Anotá de cada uno: largo, si cita, tiempo y fuentes.
- ¿Cuál dejarías en tu proyecto, y por qué?

Repaso: hoy te llevás tres ideas

Framework

piezas armadas, pasos escondidos

Memoria

reenviar la conversación

Medir

igual, con cualquier librería

Un hilo por persona: cada conversación guarda solo lo suyo.

¿Qué ya no podrías explicar?

Nombrá una parte de tu sistema que ahora hace LangChain y que no sabrías explicar sin leer su código.

Anotá tu **confianza (1 a 4)**.